

Hệ quy chiếu và hệ tọa độ trong tin học

Wang Ye

Trường Trung học số 1 Vu Hồ, An Huy

2006

Nguồn gốc: Hệ quy chiếu và hệ tọa độ trong tin học

IOI China candidate team papers / 2006 / Wang Ye.doc

Tóm tắt

Tin học là một ngành còn trẻ so với các ngành kinh điển như toán học và vật lý, nên nhiều tư tưởng trong tin học bắt nguồn từ những ngành ấy. Bài viết giới thiệu ứng dụng của hệ quy chiếu và hệ tọa độ trong tin học qua ba góc nhìn: thay đổi đơn vị độ dài, chọn hệ quy chiếu, và xây dựng hệ tọa độ. Mục đích là mở rộng cách suy nghĩ khi giải bài, biến các tư tưởng này thành công cụ hữu ích trong thi lập trình.

Từ khóa: tin học; hệ quy chiếu; hệ tọa độ; đơn vị độ dài; rời rạc hóa; trục số.

1 Dẫn nhập

Trong quá trình làm bài, đôi khi ta bất chợt nghĩ ra một cách giải rất đặc biệt. Những ý tưởng như vậy thường để lại ấn tượng mạnh, nhưng không thể chỉ trông chờ vào cảm hứng. Khi gặp bài lạ, ta vẫn cần những phương pháp phân tích có hệ thống.

Bài viết xem tư tưởng hệ quy chiếu và hệ tọa độ như một ngọn đèn chỉ đường. Phần đầu nhắc lại khái niệm cơ bản; phần sau trình bày ba ứng dụng: thay đổi ý nghĩa đơn vị độ dài, chọn hệ quy chiếu thích hợp, và xây dựng hệ tọa độ mới. Mỗi phần đều bắt đầu từ một bài toán, phân tích từ cách nghĩ thông thường tới cách nghĩ được tối ưu hóa.

2 Khái niệm cơ bản

2.1 Hệ quy chiếu

Khi nói cây cối hay nhà cửa đứng yên, còn ô tô đang chạy, ta đang lấy mặt đất làm chuẩn. Nhưng một hành khách ngồi trong tàu hỏa lại có thể xem mình đứng yên và cây bên đường lùi về sau. Khi mô tả chuyển động, vật được chọn làm chuẩn gọi là **hệ quy chiếu**.

Trong giải bài, “hệ quy chiếu” có thể hiểu rộng hơn: ta chọn một đại lượng chuẩn để mô tả các đại lượng khác. Nếu chuẩn được chọn khéo, quan hệ giữa các đại lượng trở nên đơn giản.

2.2 Hệ tọa độ

Trên một vật làm chuẩn, chọn một điểm gốc O , các hướng dương và đơn vị độ dài, ta có thể định lượng vị trí bằng tọa độ. Ba yếu tố cơ bản của một hệ tọa độ là:

- gốc tọa độ;
- hướng dương;
- đơn vị độ dài.

Trong hệ tọa độ một chiều, mỗi điểm trên trục số được biểu diễn bằng một số. Trong hệ tọa độ hai chiều quen thuộc, một điểm được biểu diễn bằng cặp (x, y) . Trong hệ tọa độ ba chiều, một điểm được biểu diễn bằng bộ (x, y, z) . Ý tưởng cốt lõi là thiết lập tương ứng giữa đối tượng và một bộ số có thứ tự, sao cho các thao tác hình học hoặc trạng thái của bài toán có thể được mô tả bằng phép toán trên số.

3 Thay đổi đơn vị độ dài

3.1 Ý tưởng

Với cùng một dữ liệu, chọn ý nghĩa khác nhau cho đơn vị độ dài sẽ tạo ra hệ tọa độ khác nhau và làm độ khó bài toán thay đổi. Ví dụ, nếu biểu diễn điểm thì trên trục số thì đơn vị tự nhiên là một điểm số; nếu biểu diễn thứ hạng, đơn vị là một thứ hạng. Việc chuyển từ điểm số sang thứ hạng chính là một dạng rời rạc hóa.

Rời rạc hóa thường dùng khi miền giá trị rất lớn nhưng số lượng giá trị thật sự xuất hiện nhỏ. Khi đó, thay vì dùng tọa độ gốc, ta dùng thứ tự của các tọa độ có ý nghĩa.

3.2 Ví dụ: AHOI 2005 Cross

Bài *Cross* có một robot cần đi từ điểm xuất phát tới quặng trong một mặt phẳng có N vùng từ trường hình vuông, các cạnh song song với trục tọa độ. Robot chỉ được đi ngang hoặc dọc, không được đi dọc biên từ trường. Mỗi lần vượt qua biên từ trường tiêu tốn một lần năng lượng; cần cực tiểu số lần vượt biên.

Cách nghĩ đầu tiên là lấy mọi điểm nguyên trong miền tọa độ làm đỉnh, nối các đỉnh kề nhau bằng cạnh trọng số 0 hoặc 1 tùy có vượt biên hay không, rồi chạy Dijkstra. Nhưng miền tọa độ lên tới 10000×10000 , nên số đỉnh quá lớn.

Cải tiến bằng heap chỉ giảm chi phí chọn đỉnh nhỏ nhất, nhưng số đỉnh vẫn quá lớn. Điểm đặc biệt của bài là số tọa độ có ý nghĩa nhỏ: tọa độ robot, tọa độ quặng, và các tọa độ cạnh của các hình vuông. Với $N < 100$, mỗi chiều chỉ có khoảng $2N + 2$ tọa độ quan trọng. Ta lấy các tọa độ này, sắp xếp, rồi dùng thứ tự của chúng làm tọa độ mới. Lúc này mỗi đơn vị trên trục không còn là “độ dài 1” trong hình gốc, mà là “khoảng giữa hai tọa độ quan trọng kề nhau”. Số đỉnh giảm xuống khoảng 204^2 , đủ nhỏ để giải.

Sau rời rạc hóa, có thể tiếp tục tối ưu bằng cách tô màu các miền liên thông không cắt biên từ trường. Khi đó bài toán trở thành tìm đường ngắn nhất giữa hai miền màu, mỗi lần đi qua biên giữa hai miền có chi phí 1. Với số ô sau rời rạc hóa là V , độ phức tạp có thể đưa về $\mathcal{O}(NV)$ hoặc tốt hơn tùy cách dựng quan hệ kề.

3.3 Tiểu kết

Trong *Cross*, lúc đầu ta dùng đơn vị độ dài gốc của đề bài. Sau đó, ta đổi đơn vị thành thứ tự của các tọa độ hiệu lực. Đây là rời rạc hóa, nhưng cũng có thể xem là thay đổi ý nghĩa của đơn vị độ dài trong hệ tọa độ. Khi hệ tọa độ gốc làm không gian quá lớn, cần nghĩ tới việc thiết kế lại đơn vị cho phù hợp với thông tin thật sự quan trọng.

4 Chọn hệ quy chiếu

4.1 Ý tưởng

Quan sát cùng một chuyển động trong các hệ quy chiếu khác nhau có thể cho mô tả khác nhau. Nếu nhiều vật cùng chuyển động với vận tốc như nhau, lấy một trong chúng làm hệ quy chiếu sẽ biến nhiều chuyển động thành đứng yên, và chỉ còn một vài đại lượng thay đổi.

Trong thuật toán, điều này tương ứng với việc thay đổi chuẩn so sánh. Nếu ta chỉ cần quan hệ tương đối, không nhất thiết phải duy trì giá trị tuyệt đối.

4.2 Ví dụ: Pudding

Bài *Pudding* cho N tuần. Tuần i có chi phí sản xuất một đơn vị là C_i , nhu cầu là P_i . Hàng có thể lưu kho với chi phí S mỗi đơn vị mỗi tuần, nhưng chỉ được lưu tối đa T tuần. Cần thỏa mọi nhu cầu với tổng chi phí nhỏ nhất.

Với tuần i , nếu sản xuất ở tuần $j \leq i$, $i - j \leq T$, đơn giá đến khi bán là

$$C_j + S(i - j).$$

Vì nhu cầu P_i đã biết, ta cần tìm

$$V_i = \min_{i-T \leq j \leq i} \{C_j + S(i - j)\}.$$

Tính trực tiếp mất $\mathcal{O}(NT)$, không đủ với $N, T \leq 40000$.

Nếu dùng heap để lưu giá trị thật của từng lô hàng ở tuần hiện tại, mỗi tuần mọi giá trị trong heap đều phải cộng thêm S , thao tác này không tiện. Ta đổi hệ quy chiếu: thay vì lưu giá trị thật $C_j + S(i - j)$, lưu

$$C_j - Sj.$$

Khi xét tuần i ,

$$C_j + S(i - j) = (C_j - Sj) + Si.$$

Hạng Si giống nhau với mọi j , nên thứ tự so sánh chỉ phụ thuộc vào $C_j - Sj$. Như vậy ta không cần cộng S cho toàn bộ heap mỗi tuần; giá trị mới được chèn là $C_i - Si$, và giá trị nhỏ nhất thật sự được khôi phục bằng cách cộng Si .

Sau bước đổi hệ quy chiếu này, bài toán còn là tìm minimum trong cửa sổ trượt dài T . Dùng heap được $\mathcal{O}(N \log T)$; dùng hàng đợi đơn điệu có thể đạt $\mathcal{O}(N)$:

1. xóa khỏi đầu hàng đợi các tuần đã quá hạn;
2. trước khi chèn $C_i - Si$, xóa khỏi cuối hàng đợi các giá trị không nhỏ hơn nó;
3. đầu hàng đợi là tuần sản xuất rẻ nhất cho nhu cầu hiện tại.

4.3 Tiểu kết

Với các bài chỉ cần so sánh tương đối, có thể thay giá trị thật bằng giá trị trong một hệ quy chiếu khác. Trong Pudding, mọi giá trị cũ cùng tăng S mỗi tuần; lấy xu thế tăng ấy làm chuẩn, ta biến thao tác cập nhật hàng loạt thành thao tác chèn một giá trị đã trừ sẵn.

5 Xây dựng hệ tọa độ

5.1 Ý tưởng

Thông thường, hệ tọa độ d chiều cần d trục độc lập để biểu diễn mọi điểm. Nhưng hệ tọa độ được đề bài gợi ý chưa chắc là hệ tọa độ tốt nhất. Nếu đối tượng có cấu trúc đặc biệt, ta có thể tạo hệ tọa độ mới để các bước di chuyển hoặc khoảng cách trở nên đơn giản.

5.2 Ví dụ: Delta-wave

Bài *Delta-wave* đánh số các ô tam giác bằng các số nguyên dương tăng dần theo từng hàng. Cho hai số $M, N \leq 10^9$, cần tìm số cạnh ít nhất phải đi qua để từ ô này tới ô kia.

Cách trực tiếp xem mỗi tam giác là một đỉnh và chạy BFS là không thể, vì số ô có thể rất lớn. Dùng hàng và cột của tam giác có thể mô phỏng đường đi tốt hơn, nhưng vẫn còn phải xử lý nhiều trường hợp chắn lẻ.

Quan sát sâu hơn, lưới tam giác có ba hướng cơ bản. Vì vậy thay vì ép vào hệ hai tọa độ hàng-cột, ta xây dựng ba tọa độ tương ứng với ba họ đường song song. Với một ô được đánh số p , trước hết xác định hàng

$$r = \lfloor \sqrt{p-1} \rfloor + 1.$$

Từ vị trí trong hàng, tính ba đại lượng (x, y, z) sao cho mỗi lần đi qua một cạnh chỉ làm thay đổi một trong ba quan hệ tọa độ. Bản gốc đưa ra công thức tương đương trong chương trình:

$$\begin{aligned}x &= \lfloor \sqrt{p-1} \rfloor + 1, \\y &= \left\lfloor \frac{p - (x-1)^2 + 1}{2} \right\rfloor, \\z &= \left\lfloor \frac{x^2 - p}{2} \right\rfloor + 1.\end{aligned}$$

Khi đó khoảng cách giữa hai ô có tọa độ (x_1, y_1, z_1) và (x_2, y_2, z_2) là

$$|x_1 - x_2| + |y_1 - y_2| + |z_1 - z_2|.$$

Hệ tọa độ mới biến một bài tìm đường trên đồ thị khổng lồ thành phép tính trực tiếp $\mathcal{O}(1)$. Đây là ví dụ rõ nhất cho việc xây dựng hệ tọa độ phù hợp thay vì chấp nhận hệ tọa độ do cách đánh số ban đầu áp đặt.

6 Kết luận

Ba ví dụ trên đều thể hiện cùng một tư tưởng: cách mô tả đối tượng quyết định độ khó của bài toán. Thay đổi đơn vị độ dài giúp rời rạc hóa không gian lớn; chọn hệ quy chiếu tốt giúp loại bỏ cập nhật đồng loạt; xây dựng hệ tọa độ mới giúp biến đường đi phức tạp thành công thức đơn giản.

Khi gặp bài toán lạ, ngoài việc tìm thuật toán trực tiếp, nên tự hỏi: hệ tọa độ hiện tại có đang làm bài toán phức tạp hơn không? Có đại lượng chuẩn nào giúp ta chỉ còn xét quan hệ tương đối không? Có cách biểu diễn khác khiến bước chuyển trạng thái trở nên tự nhiên hơn không? Trả lời được những câu hỏi ấy thường mở ra một hướng giải ngắn gọn hơn.

Ghi chú về phụ lục

Bản gốc kèm nhiều chương trình cho các phiên bản của Cross, Puddin và Delta-wave, cùng các đề bài gốc. Vì đây là các đoạn mã mẫu dài và người dùng không yêu cầu dịch code template, bản dịch chỉ giữ lại ý tưởng thuật toán, công thức và độ phức tạp cần thiết.

Lời cảm ơn

Tác giả cảm ơn thầy Jiang Tao đã giúp đỡ trong quá trình chuẩn bị bài viết, và cảm ơn Zhu Chenguang, Zhou Dong vì các ý kiến góp ý.